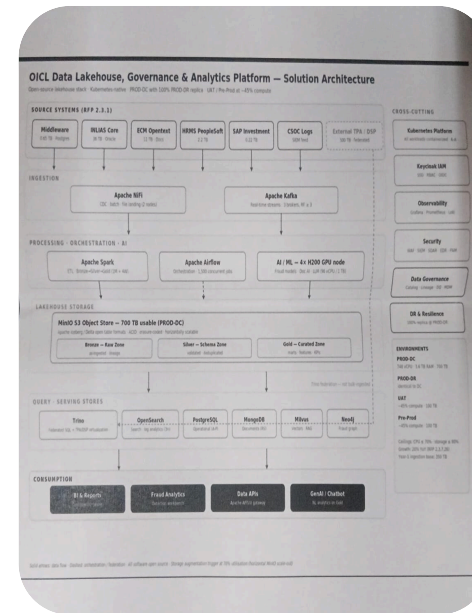




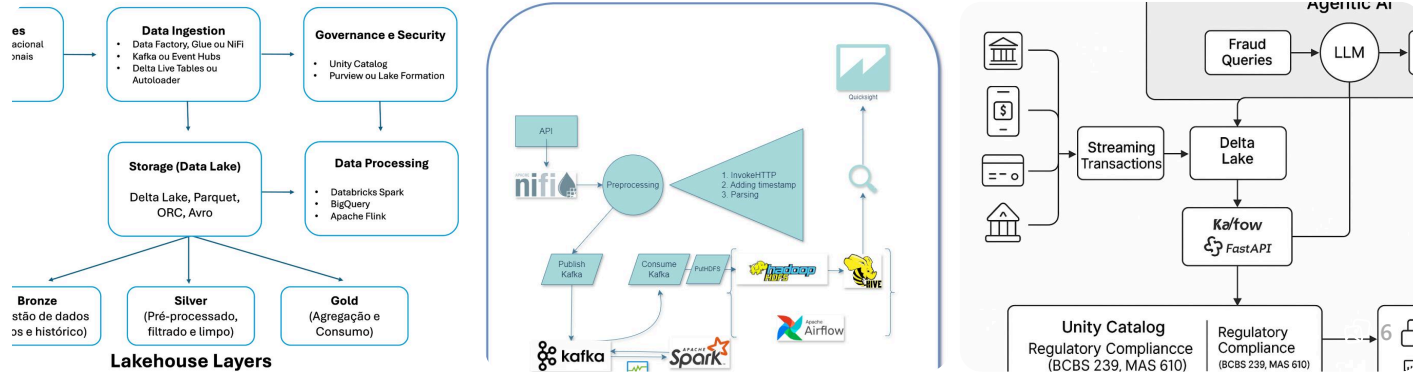
Explain this architecture in detail in simple words

Worked for 11s >

Absolutely. This diagram is essentially showing how OICL's data will travel from different source systems → into the Data Lakehouse → get processed/governed → become available for analytics, fraud detection, APIs and GenAI.

Think of it like a large central data factory:

Sources → Ingestion → Processing → Storage → Query/Serving → Business Applications



1. First understand the complete flow

The architecture can be simplified to:



And **across all these layers**, there are common services such as:

- Kubernetes
- IAM
- Observability
- Security
- Data Governance

- DR & Resilience

These are called **cross-cutting services** because they don't belong to just one layer—they support the entire platform.

2. SOURCE SYSTEMS

At the very top, we have the systems from which OICL's data originates.

The diagram shows:

Source	What it means
Middleware	Integration/middleware systems
INLIAS Core	OICL core insurance/application system
ECM OpenText	Enterprise Content Management
HRMS PeopleSoft	Human Resources data
SAP Investment	Investment/financial data
CSOC Logs	Cyber/security operation logs
External TPA/DSP	External third-party data

Simple example

Suppose an OICL policy is created.

The policy information may initially exist in the **core insurance system**.

Similarly:

- Employee information → HRMS
- Investment information → SAP

- Documents → OpenText
- Security events → CSOC
- Third-party claims/services → TPA/DSP

The Data Lakehouse needs to bring all this information together.

That's where **Ingestion** starts.

3. INGESTION LAYER

The diagram has two major technologies here:

Apache NiFi

Apache Kafka

They solve slightly different problems.

3.1 Apache NiFi

NiFi is used for **moving data from source systems into the platform**.

Think of NiFi as a **data transportation and routing system**.

For example:



NiFi can handle:

- Batch data
- File-based data
- Database data

- CDC
- Data routing
- Data transformation
- Data movement

What is CDC?

CDC = **Change Data Capture**.

Suppose an employee's department changes:

Before:

Employee 101 → Finance

After:

Employee 101 → IT

Instead of sending the entire HR database again, CDC can capture the change:

Employee 101

Department changed

Finance → IT

That is much more efficient.

4. Apache Kafka

Kafka is primarily for **real-time streaming data**.

Think of Kafka as a **real-time conveyor belt**.

For example:

Application



Kafka



Processing



Fraud Detection

Suppose an insurance claim is submitted.

Kafka can immediately stream the event:

Claim submitted



Kafka



Fraud model



Risk = HIGH

So:

NiFi

Good for:

Moving, routing and ingesting different types of data, including batch/file/CDC.

Kafka

Good for:

Continuous real-time event streaming.

They can therefore complement each other.

5. PROCESSING, ORCHESTRATION & AI

After data enters the platform, it has to be processed.

The diagram shows:

- Apache Spark

- Apache Airflow
- AI/ML – 4 × H200 GPU node

6. Apache Spark

Spark is the **heavy-duty data processing engine**.

Imagine OICL has:

500 million policy/claim/customer records.

Spark can process huge amounts of data in parallel.

For example:

```
Raw data
  ↓
Spark
  ↓
Clean
Transform
Join
Aggregate
Calculate
  ↓
Processed data
```

Example

Suppose:

```
Policy table
Claims table
Customer table
```

Spark could join them and calculate:

Customer
Total Policies
Total Claims
Total Claim Amount
Claim Frequency
Risk Score

This processed data then goes into the Lakehouse layers.

7. Apache Airflow

Airflow is different from Spark.

A very important distinction:

Spark does the work. Airflow decides when and in what order the work happens.

Think of Airflow as the **manager/scheduler**.

For example:

```
graph TD
    A[12:00 AM] --> B[Extract data]
    B --> C[Validate data]
    C --> D[Run Spark job]
    D --> E[Load Silver]
    E --> F[Create Gold tables]
    F --> G[Update analytics]
```

Airflow manages this workflow.

It can also:

- Schedule jobs
- Monitor jobs
- Retry failed jobs
- Define dependencies
- Run pipelines
- Coordinate multiple processing tasks

Example

You could have:

```
Task 1 → Ingest claims
      ↓
Task 2 → Validate claims
      ↓
Task 3 → Spark transformation
      ↓
Task 4 → Store Silver
      ↓
Task 5 → Create Gold dataset
```

Airflow makes sure the tasks happen in the correct sequence.

8. AI / ML – 4 × H200 GPU Nodes

This is the **AI/ML compute section**.

The architecture proposes:

4 × NVIDIA H200 GPU nodes

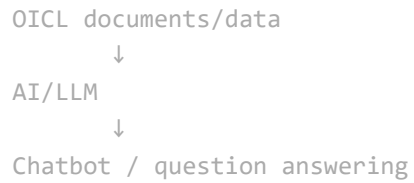
These GPUs are intended for computationally intensive AI/ML workloads.

For OICL, examples could include:

Fraud detection



GenAI



Other possible workloads

- Fraud prediction
- Anomaly detection
- Classification
- Risk scoring
- NLP
- Document intelligence
- Embeddings
- GenAI/LLM inference

So the GPU infrastructure is essentially the **AI engine** of the platform.

9. LAKEHOUSE STORAGE

This is probably the **most important section** of the architecture.

The diagram proposes:

MinIO S3 Object Store – 700 TB usable (PROD-DC)

MinIO provides **S3-compatible object storage**.

Think of this as the **main warehouse where the organization's data is stored**.

10. Why is it called a Data Lakehouse?

A traditional:

Data Warehouse

is structured and optimized for analytics.

A:

Data Lake

can store many types of raw data.

A:

Data Lakehouse

tries to provide the flexibility of a data lake with many of the management/analytics capabilities associated with a warehouse.

This architecture is therefore using object storage as the central data foundation.

11. Bronze → Silver → Gold

This is extremely important.

The architecture divides the data into three zones.

Bronze – Raw Zone

This is the **original/raw data**.

Example:

Source:

HRMS

Data:

Employee_ID | Name | Dept | Salary

The idea is:

Don't heavily modify the original data.

Bronze acts as the raw historical copy.

Think:

Bronze = What came from the source

12. Silver – Schema Zone

Now the raw data gets:

- Cleaned
- Validated
- Standardized
- Deduplicated
- Transformed
- Structured

For example:

Bronze:

"01-08-26"

"2026/08/01"

"Aug 1 2026"

Silver could standardize these into:

```
2026-08-01
```

Similarly:

```
"Male"
```

```
"M"
```

```
"M"
```

could be standardized.

Think:

Silver = Clean and standardized data

13. Gold – Curated Zone

Gold contains **business-ready data**.

For example, instead of making a BI dashboard repeatedly process millions of raw records, we can create a curated dataset:

```
Monthly Claims Summary
```

Month	Claims	Amount	Fraud Cases
Jan	20,000	₹XX	350
Feb	22,000	₹XX	420

This is much easier for:

- BI
- Reporting
- Fraud analytics
- APIs
- Business users

Think:

Gold = Ready-to-use business data

14. Very simple example of the three layers

Imagine a customer makes an insurance claim.

Bronze

Raw claim received

↓

Silver

Claim cleaned
Customer ID validated
Date standardized
Duplicate removed
Amount validated

↓

Gold

Customer claim history
Claim frequency
Claim amount
Fraud risk
Business KPIs

That's the basic idea behind:

Bronze → Silver → Gold

15. QUERY & SERVING STORES

After data is stored, different applications need to **access it efficiently**.

That's why the architecture doesn't use one database for everything.

It has:

- Trino
- OpenSearch
- PostgreSQL
- MongoDB
- Milvus
- Neo4j

Each has a different purpose.

16. Trino

Trino is the **SQL query engine**.

Think of Trino as a person who can go into different data stores and ask:

"Give me the data I need."

For example:

```
SELECT
    month,
    SUM(claim_amount)
FROM claims

SQL
```

Trino can query data stored in the lakehouse.

The architecture also indicates **federated SQL**.

That means Trino can potentially query data from multiple sources without physically moving everything into one database.

Simple example

```
MinIO
  \
PostgreSQL → Trino → SQL Query
  /
Other sources
```

17. OpenSearch

OpenSearch is designed primarily for:

- Search
- Log analytics
- Fast text-based querying
- Operational analytics

For example:

```
Search:
"Claim number 12345"
```

or:

```
Find all security logs
where severity = HIGH
```

It is particularly useful for log/search-oriented workloads.

18. PostgreSQL

PostgreSQL is a **relational database**.

It is useful for structured operational data.

For example:

```
Customer
Policy
Transaction
Application
Configuration
```

It is not necessarily intended to replace the large central lakehouse.

Instead, it can support **operational/metadata/application workloads**.

19. MongoDB

MongoDB is a **document database**.

Instead of storing data primarily as rows and columns, it works well with document-style data.

Example:

```
{
  "customer": 101,
  "policy": 5001,
  "claims": [...]
```

JSON

Useful where flexible document structures are required.

20. Milvus

Milvus is a **vector database**.

This becomes important for **AI/GenAI**.

Suppose OICL has thousands of documents:

```
Policy documents
Claim documents
Circulars
Guidelines
FAQs
Reports
```

These documents can be converted into **embeddings/vectors**.

Milvus stores those vectors.

Then a user can ask:

```
"What is the procedure for settling this type of claim?"
```

The system can find documents that are **semantically similar** to the question.

This is commonly used in:

```
RAG – Retrieval Augmented Generation
```

21. Neo4j

Neo4j is a **graph database**.

It is particularly useful when relationships between entities are important.

For fraud detection, imagine:

Customer A
↓
Phone Number X
↓
Bank Account Y
↓
Customer B
↓
Policy Z

A graph database can help identify suspicious relationships.

For example:

Multiple customers
↓
Same phone number
↓
Same bank account
↓
Multiple suspicious claims

That can be extremely useful for **fraud analytics**.

22. Why so many databases?

This is an important architecture concept.

The design is **polyglot persistence**.

It means:

Don't force every workload into one database.

Instead:

Technology	Main purpose
MinIO	Central lakehouse/object storage
Trino	SQL querying
OpenSearch	Search/log analytics
PostgreSQL	Relational/operational workloads
MongoDB	Document workloads
Milvus	Vector/AI search
Neo4j	Relationship/graph analytics

Each technology is selected for a specific workload.

23. CONSUMPTION LAYER

This is where the **business users finally see/use the data**.

The diagram shows four major consumption areas:

BI & Reports

Fraud Analytics

Data APIs

GenAI / Chatbot

24. BI & Reports

BI = Business Intelligence.

Examples:

```
Claims Dashboard
Policy Dashboard
Investment Dashboard
Branch Performance
Premium Analysis
Customer Analysis
```

Business users can view:

- KPIs
- Charts
- Trends
- Reports
- Dashboards

The data generally comes from the curated/Gold layer or serving systems.

25. Fraud Analytics

This is one of the major use cases.

The flow could look like:

```
Claims
Policies
Customers
Transactions
External data
↓
Lakehouse
↓
Spark / ML
↓
Fraud models
↓
Fraud score
```

↓
Fraud Analytics

For example:

Claim #12345
Risk Score = 92%

The system can flag it for investigation.

26. Data APIs

The data shouldn't only be consumed by dashboards.

Other applications may need data.

For example:

Mobile App
↓
API
↓
Data Platform
↓
Customer information

So Data APIs provide controlled access to data for other applications/services.

27. GenAI / Chatbot

This is the AI interface.

A user could ask:

"Show me the claims with unusually high fraud risk in Karnataka."

Or:

"What is the claim settlement procedure for policy type X?"

The system can potentially combine:

Lakehouse data
+
Vector search
+
LLM
+
Business rules

and provide an answer.

28. CROSS-CUTTING SERVICES

Now we come to the **right side of the architecture**.

This section is extremely important because these services apply to almost the entire platform.

29. Kubernetes Platform

The architecture is described as:

Kubernetes-native

Kubernetes manages containerized applications.

Instead of manually managing every application server:

Server 1
Server 2
Server 3
Server 4

...

Kubernetes manages workloads as containers/pods.

For example:



It can help with:

- Deployment
- Scaling
- Restarting failed containers
- Resource management
- High availability
- Service discovery

30. Keycloak IAM

IAM = Identity and Access Management.

Keycloak handles things such as:

- User authentication
- Login
- Roles
- Permissions
- Single Sign-On

For example:

Admin

Can access:

- All data
- All configurations

Analyst

Can access:

- Analytics
- Reports

Data Engineer

Can access:

- Data pipelines
- Processing

The objective is:

- The right user should get the right access.

31. Observability

The diagram mentions:

- Grafana / Prometheus / Loki

These are used for monitoring and observability.

Think of this as the **health-monitoring system of the platform**.

You want to know:

Is Kafka running?
Is Spark running?
How much CPU is being used?
How much memory?
Are jobs failing?
Is storage filling up?
Are APIs responding?

For example:

CPU usage = 85%
Memory = 78%
Kafka broker = Healthy
Spark job = Failed

This helps operations teams detect problems quickly.

32. Security

The diagram mentions security technologies such as:

- IAM
- SIEM
- SOAR
- EDR
- PAM

These provide different security capabilities.

IAM

Controls identities and access.

SIEM

Security Information and Event Management.

Collects and analyzes security logs/events.

SOAR

Security Orchestration, Automation and Response.

Automates security responses.

EDR

Endpoint Detection and Response.

Monitors endpoints/servers for suspicious activity.

PAM

Privileged Access Management.

Controls highly privileged accounts.

So security isn't just one product—it is a **layer around the entire platform**.

33. Data Governance

The diagram mentions:

Catalog, Lineage, DQ, MDM

These are extremely important in an enterprise data platform.

Data Catalog

Answers:

What data do we have?

Example:

Customer_Master
Claims
Policies

Data Lineage

Answers:

Where did this data come from?

Example:



So if someone asks:

"Where did this KPI come from?"

lineage can help answer that.

Data Quality – DQ

Answers:

Is the data correct and usable?

Checks can include:

- Null values

- Duplicate records
- Invalid dates
- Invalid IDs
- Incorrect formats
- Missing fields

MDM

MDM = **Master Data Management**.

It tries to establish trusted master records for important entities such as:

- Customer
- Policy
- Branch
- Employee
- Product

34. DR & RESILIENCE

DR = **Disaster Recovery**.

The diagram shows:

100% replica @ PROD-DR

The basic idea is:



DC = Data Centre

DR = Disaster Recovery site.

If the primary data centre experiences a major failure, the DR environment is available to support recovery/business continuity.

35. ENVIRONMENTS

The architecture shows four environments:

PROD-DC

Production environment in the primary data centre.

This is the **live environment**.

PROD-DR

Production Disaster Recovery.

This is the recovery environment.

The architecture indicates that it is intended to be a **replica of production**.

UAT

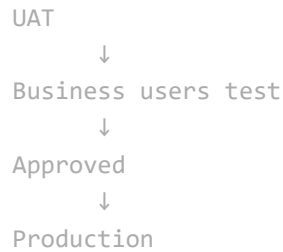
UAT = **User Acceptance Testing**.

Business users test whether the system works correctly before production deployment.

Example:

New dashboard





Pre-Prod

Pre-production is an environment that closely resembles production and is used for final technical validation before going live.

36. Why does UAT/Pre-Prod have lower compute?

At the top of the architecture, it says:

PROD-DC with 100%, PROD-DR replica, UAT / Pre-Prod at ~45% compute

The idea is that production requires the full capacity because it handles actual workloads.

UAT and Pre-Prod don't normally need the same compute capacity as production.

For example, conceptually:

Production	→ 100% compute
UAT	→ ~45%
Pre-Prod	→ ~45%

This reduces infrastructure cost while still providing sufficient capacity for testing.

37. Put everything together

Let's follow one example.

Suppose a customer submits an insurance claim.

Step 1 – Source

Claim originates from an OICL application/system.

Source System
↓

Step 2 – Ingestion

If it is an event that needs real-time processing:

Kafka

If it is batch/file/CDC-oriented:

NiFi

Step 3 – Processing

Data can be processed using:

Spark

Airflow controls the workflow:

Airflow
↓
Spark

Step 4 – Storage

Raw claim:

Bronze

Cleaned claim:

Silver

Business-ready claim dataset:

Gold

All stored on:

MinIO S3

Step 5 – Analytics

Trino can query the data.

Fraud analytics can use:

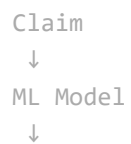
Spark + ML

Neo4j can analyze relationships.

Step 6 – AI

The AI/ML infrastructure can process the claim.

For example:



Fraud probability = 87%

Step 7 – Consumption

The result can reach:

Fraud Dashboard
+
BI Reports
+
Data APIs
+
GenAI/Chatbot

38. The most important architectural idea

If you are reviewing this architecture against an RFP, don't just look at the individual products.

Look at whether the **complete data lifecycle is covered**.

The architecture is basically covering:

Requirement area	Architecture component
Source integration	Source systems
Data ingestion	NiFi + Kafka
Batch processing	Spark
Real-time processing	Kafka + processing
Workflow orchestration	Airflow
Central data lakehouse	MinIO

Requirement area	Architecture component
Raw data	Bronze
Clean data	Silver
Curated data	Gold
SQL analytics	Trino
Search/log analytics	OpenSearch
Relational workloads	PostgreSQL
Document workloads	MongoDB
Vector/AI search	Milvus
Graph/fraud relationships	Neo4j
BI	BI & Reports
Fraud detection	Fraud Analytics + AI/ML
API access	Data APIs
GenAI	GenAI/Chatbot + GPU
Container platform	Kubernetes
IAM	Keycloak
Monitoring	Prometheus/Grafana/Loki
Security	IAM/SIEM/SOAR/EDR/PAM
Governance	Catalog/Lineage/DQ/MDM

Requirement area	Architecture component
DR	PROD-DR
Testing	UAT + Pre-Prod

39. One thing I would pay special attention to in this architecture

The diagram is a **high-level solution architecture**, not necessarily proof that every detailed RFP requirement is satisfied.

For an RFP review, I would ask the OEM/vendor to explicitly map:

RFP requirement → component/product → edition/version → quantity → capacity → HA/DR → license → environment → compliance

For example, merely writing:

"Data Governance – Catalog, Lineage, DQ, MDM"

doesn't prove that all four capabilities are actually covered.

You need to know **which product provides each capability**, its edition, deployment model, licensing and sizing.

Similarly:

"Security – IAM, SIEM, SOAR, EDR, PAM"

is a capability statement. It doesn't by itself tell you which actual products are being proposed.

40. In one sentence

The entire architecture can be understood as:

OICL collects data from many insurance/business systems, brings it into the platform using NiFi/Kafka, processes it using Spark and Airflow, stores it centrally in MinIO using Bronze/Silver/Gold layers, makes it available through specialized query/serving databases, and finally delivers BI, fraud analytics, APIs and GenAI—with Kubernetes, security, governance, monitoring and DR protecting the entire platform.

If you're using this for your **OICL RFP technical review**, the next useful step is to break this exact diagram into **Data Lake + Data Warehouse/Lakehouse + Data Governance + Fraud Detection + BI + AI/ML + Infrastructure + Security**, and identify what is covered, what is missing, and what questions you should ask the OEM/vendor.



Explain the flow in short and simple

Sure. The **whole flow** can be understood in just 6 simple steps:

OICL Data Flow

1. Source Systems → Data comes from different OICL systems

Examples:

- INLIAS
- HRMS
- SAP
- OpenText
- CSOC
- External systems



2. Ingestion → Bring the data into the platform

- **NiFi** → batch/files/CDC data
- **Kafka** → real-time streaming data



3. Processing → Clean and process the data

- **Spark** → processes/transforms large data
- **Airflow** → schedules and manages the processing jobs
- **AI/ML** → performs AI/fraud-related processing



4. Lakehouse Storage → Store the data

Data is stored in **MinIO** in 3 layers:

Bronze → **Silver** → **Gold**

-  **Bronze** = Raw data
-  **Silver** = Cleaned/processed data
-  **Gold** = Business-ready data



5. Query & Serving → Make the data easily usable

Different tools serve different purposes:

- **Trino** → SQL queries
- **OpenSearch** → search/logs
- **PostgreSQL/MongoDB** → application data
- **Milvus** → AI/vector search
- **Neo4j** → relationships/fraud analysis



6. Consumption → Business users use the data

Finally, the data goes to:

- 📊 BI & Reports
- 🚨 Fraud Analytics
- 🔌 Data APIs
- 🤖 GenAI/Chatbot

In one line:

Source Systems → NiFi/Kafka → Spark/Airflow → MinIO (Bronze/Silver/Gold) → Trino & other serving stores → BI/Fraud/API/GenAI

And Kubernetes, Security, IAM, Monitoring, Governance and DR support the entire flow from beginning to end.



ChatGPT can make mistakes. Check important info.